

Prompting Correcto para LLMs (genérico, con foco en programación)

Ingeniería Civil Informática + Proyectos de software

Plantilla didáctica para estudiantes

16 de abril de 2026

Objetivo de la guía

Que aprendas

- Cómo escribir prompts que funcionen con **cualquier LLM**.
- Cómo usar prompts en **VS Code** / **Cursor** para mejorar resultados en programación.
- Cómo mantener consistencia con un archivo opcional `conventions.md`.

Idea central

Un buen prompt es **específico, accionable y verificable**.

¿Por qué importa el prompting?

Sin prompting (malo)

- “Arregla esto” sin contexto.
- “Hazlo bien” sin criterios de calidad.
- Respuestas genéricas y difíciles de evaluar.

Con prompting (bueno)

- Contexto claro (repo, lenguaje, restricciones).
- Tarea concreta (plan, diff, tests, etc.).
- Criterios de aceptación (“cómo sabrás que está bien”).

Un prompt completo suele tener

- **Rol:** “Actúa como tutor/revisor/arquitecto”.
- **Contexto:** tema, enunciado, lenguaje, entorno y archivos relevantes.
- **Tarea:** qué quieres que el LLM produzca.
- **Restricciones:** librerías permitidas, estilo, límites, etc.
- **Criterios de aceptación:** condiciones verificables.
- **Formato de entrega:** secciones exactas, orden, si es diff o archivo completo.
- **Iteración:** qué hacer si falta información.

Cláusula anti-“alucinar” (muy útil)

Pídelo explícitamente

Antes de escribir mucho código:

- 1) Identifica qué información falta.
- 2) Pregunta antes de asumir.
- 3) Si propones un plan, espera confirmación.

Beneficio

Reduce suposiciones incorrectas y mejora la calidad del proceso.

Plantilla: solicitud general (Parte 1)

Rol: Actúa como tutor experto en Ingeniería Civil Informática y desarrollo de software.

Contexto:

- Proyecto/Asignatura: <<...>>
- Objetivo: <<...>>
- Stack: <<lenguaje/version/entorno>>
- Restricciones: <<p.ej. sin librerías X, estilo Y>>
- Archivos relevantes (si aplica): <<...>>
- Observaciones: <<logs/errores/decisiones previas>>

Tarea:

- 1) <<primera cosa>>
- 2) <<segunda cosa>>

Plantilla: solicitud general (Parte 2)

Conventions (si aplica):

- Lee y respeta conventions.md.
- Si no lo tienes, pregunta o pide que lo pegue.

Restricciones:

- No inventes APIs/dependencias.
- Si falta información, pregunta primero.

Entregable (formato exacto):

- a) Resumen corto (3-5 líneas)
- b) Plan (lista numerada)
- c) Cambios: diff o archivos/fragmentos
- d) Cómo probar (comandos/casos)
- e) Riesgos / trade-offs

Caso típico 1: plan antes de codificar

Prompt sugerido

- “Propón una descomposición en módulos/clases”.
- “Define estrategia de tests”.
- “Indica dependencias internas y riesgos”.

Regla

Si el plan es incorrecto, el código también lo será: valida temprano.

Caso típico 2: bug con logs

Actúa como ingeniero de debugging.

Contexto:

- Lenguaje/stack: <<...>>
- Error/logs completos: <<pega el stack trace completo>>
- Código relevante: <<pega solo los archivos/funciones necesarias>>

Tarea:

- 1) Lista hipótesis de causa (ordenadas por probabilidad).
- 2) Señala exactamente qué funciones/líneas afectan.
- 3) Propón una corrección mínima y segura.
- 4) Agrega/ajusta tests si aplica.

Entregable:

- Cambios concretos (diff o archivos)
- Explicación breve: por qué falla y por qué la corrección sirve

Caso típico 3: code review estilo tutor

Tarea concreta que debes pedir

- Señalar problemas de diseño, legibilidad, complejidad y robustez.
- Priorizar hallazgos: “crítico” vs “mejorable”.
- Sugerir mejoras incrementales con ejemplo de cómo quedaría.

Entrega recomendada

“Hallazgos + propuesta de cambios listos para aplicar”.

El prompting rinde al máximo con comprensión profunda

Idea clave

El LLM no “entiende” tu problema como tú: **el entendimiento lo construyes tú** a partir de enunciado, restricciones y validación.

A más contexto real, más útil el prompt

Mientras mejor describas entradas/salidas, bordes, supuestos y criterios de éxito, el modelo puede proponer soluciones más completas y menos improvisadas.

Qué incluir cuando realmente entendiste el problema

Checklist para que el prompt sea más completo

- **Alcance:** qué incluye y qué NO incluye la tarea.
- **Contratos:** entradas/salidas, formato y reglas del sistema.
- **Restricciones:** complejidad, librerías permitidas, estilo, límites.
- **Casos borde:** entradas raras, vacías, máximos, errores esperados.
- **Criterio de éxito:** cómo verificar “está bien” (tests, métricas).
- **Supuestos:** qué asumes y por qué (para que el LLM no adivine).

Regla de oro

Si no entiendes una parte, **pide aclaraciones** antes de pedir implementación.

¿Qué es 'conventions.md' y para qué sirve?

Sirve para

- Mantener **estilo consistente** (nombres, estructura, comentarios).
- Respetar **reglas del curso/proyecto** (p.ej. “no nuevas librerías”).
- Reducir variabilidad entre respuestas de estudiantes.
- Hacer que el LLM priorice **formato de entrega** del curso.

Consejo

Mantén el archivo **corto y accionable**.

Ejemplo de 'conventions.md' (corto)

conventions.md (ejemplo)

Idioma:

- Responde en español.

Estilo de código:

- Nombres descriptivos, sin abreviaturas.
- Manejo de errores consistente.

Formato de entregables:

- Si pido código: incluye diff si es grande.
- Si pido tests: agrega casos normales, borde y error.
- No agregues dependencias nuevas sin pedírmelo.

Cómo usar 'conventions.md' en tu prompt

Dos formas

- Si el asistente puede leer archivos: "Conventions: lee y respeta 'conventions.md'".
- Si no tiene acceso: pega el contenido o un resumen y entonces "Conventions (resumen):
...".

Regla práctica

Si hay contradicciones: pide al LLM **priorizar** las convenciones del curso.

Prompting cuando trabajas con repo

Qué pedirle al LLM en VS Code / Cursor

- “Usa la estructura del repo y respeta el diseño existente”.
- “Entrega cambios como diff por archivos”.
- “Si propones una ruta, indica por qué y cómo probar”.
- “Respetar ‘conventions.md’ si existe”.

Cláusula recomendada

“No cambies cosas que no sean necesarias; mínima modificación”.

Mini-prompt listo para Cursor/VS Code

Actúa como tutor de Ingeniería Civil Informática.
Usa el código del repo y respeta `conventions.md`.

Objetivo: <<descríbelo>>

Restricciones:

- <<p.ej. sin dependencias nuevas>>

Tarea:

- 1) Propón un plan corto (2-4 pasos).
- 2) Si es aceptado, implementa cambios mínimos.
- 3) Agrega tests y comandos para verificarlos.

Entregable:

- Resumen
- Plan
- Diff por archivos
- Cómo probar

Cursor: instrucciones persistentes del proyecto

Dónde se guardan

- Recomendado: `.cursor/rules/` (múltiples archivos, versionados)
- Alternativa simple: `AGENTS.md` en la raíz del repo
- Legacy (menos recomendado): `.cursorrules` en la raíz

Idea

Estas instrucciones se cargan como **contexto persistente** para el modelo dentro del proyecto.

Cursor: ejemplo de 'conventions.md'

```
---  
description: "Convenciones de estilo y formato para el curso"  
globs: ["**/*"]  
alwaysApply: true  
---  
  
Idioma de la explicación: español.  
Cuando pidas código: entrega `diff` por archivos si el cambio es grande.  
Prohibido agregar dependencias nuevas sin confirmación explícita.  
Siempre agrega (o ajusta) tests cuando existan.
```

Cómo usarlo

En el prompt, puedes decir: "Respetar las conventions.md del proyecto".

VS Code (Copilot Chat): custom instructions

Archivo típico (del workspace)

- `.github/copilot-instructions.md` en la raíz del proyecto

Ejemplo de contenido

“Responde en español. Pide criterios de aceptación antes de implementar. No inventes APIs.”

Nota

Si usas otra extensión (p.ej. Continue), suele tener su propia carpeta de reglas.

Prompt completo vs prompt incompleto

Incompleto (malo)

Arregla el proyecto para que funcione.

Riesgo

- No sabes qué modificar, cómo validar ni qué restricciones respetar.
- Aumenta el riesgo de cambios innecesarios o incorrectos.

Completo (bueno)

Actúa como tutor experto en Ingeniería Civil Informática.

Contexto:

- Proyecto: <<nombre>>
- Stack: <<lenguaje/version>>
- Enunciado: <<resumen>>
- Restricciones: respeta `conventions.md`

Tarea:

- 1) Proponer un plan corto (2-4 pasos).
- 2) Implementar cambios mínimos.
- 3) Agregar/ajustar tests.

Criterios de aceptación:

- Tests pasan
- Cumple el enunciado

Si falta información, pregunta antes de asumir.

LLM como asistente, no como "autor" del criterio

Qué hace bien un LLM

- Acelera tareas repetitivas: boilerplate, resúmenes, refactors mecánicos.
- Sugiere opciones y "siguientes pasos".
- Ayuda a redactar y estructurar prompts.

Lo que NO delegas

La corrección final y el criterio (supuestos, riesgos, trade-offs, revisión de PR y tests) sigue siendo **tu responsabilidad humana**.

Práctica recomendada

- 1 Pide al LLM un plan y **criterios de aceptación**.
- 2 Verifica con ejecución: **tests**, casos borde y validación contra el enunciado.
- 3 Revisa el código como tutor: “¿Esto realmente resuelve el problema?”.
- 4 Cuando haya ambigüedad, **haz preguntas** o consulta al enunciado/documentación.

Regla simple

“El LLM propone; nosotros confirmamos”.

Checklist rápido

Evita prompts débiles

- “Hazlo bien” sin criterios.
- Sin contexto (lenguaje, enunciado, archivos).
- Sin formato de entrega (qué se espera exactamente).
- Sin cómo probar o verificar.
- Sin iteración (no pedir preguntas si falta info).

Regla de oro

Si el resultado no es **verificable**, es casi seguro que será discutible.

En una frase

Escribe prompts con **contexto + tarea + restricciones + criterios + formato**.

Para tus próximos trabajos

Antes de codificar: pide plan y valida supuestos.

¿Preguntas?